Análise dos problemas da II Maratona de Programação do Norte
Gabriel Duarte

Autores da Prova: *Cristhian Bonilha - UTFPR, Francisco Elio Parente Arcos Filho - UEA, Gabriel Duarte - UFF*

Problema A - ICCANOBIF
Autor: Francisco Elio Parente Arcos Filho
Dificuldade: Fácil
Primeiro time a resolver: CCL UEA-EST 01
Tema(s): Adhoc, simulação

Nesse problema era só calcular os **N** primeiros números de Fibonacci e imprimir na ordem invertida.

Complexidade: $O(N)$

Problema B - Brincando com Consultas
Autor: Gabriel Duarte
Dificuldade: Médio/Difícil
Primeiro time a resolver: -
Tema(s): Estruturas de dados, *Sqrt-Decomposition*

É fácil ver que a solução ingênua onde para cada consulta varremos o intervalo e contamos quantas vezes o elemento **W** aparece para ter como resposta $r - l + 1 - totalVezesW$ é inviável, isso deixaria uma complexidade $O(N)$ por consulta que dado o limite do problema seria algo em torno de 10^{10} operações. Porém, podemos partir dessa solução mais simples para melhorar um pouco o tempo gasto.

A ideia é pré-calcular a quantidade de vezes que cada elemento aparece um subarray de tamanho $\sqrt{n}$. Assim, o array original terá $\sqrt{n}$ blocos e cada um deles $\sqrt{n}$ elementos. Com isso conseguimos pensar em algo mais inteligente, pois se em uma consulta um bloco está totalmente contido no intervalo, não precisamos iterar pelos elementos e contar quantas vezes o **W** aparece, basta pegar o valor que já foi pré-calculado e podemos saltar todos aqueles elementos, resolvendo cada bloco em tempo $O(X)$, onde **X** é o tempo que é gasto para saber quantas vezes um determinado elemento aparece. Iremos falar melhor sobre o **X** mais pra frente. Tendo essa solução, sabemos que no pior caso teremos que "saltar" no máximo $\sqrt{n}$ blocos para fazer uma consulta, o que gera $O(\sqrt{n} * X)$. Porém, ainda existe o caso de um bloco não está todo contido na consulta, mas nesse caso podemos fazer da maneira mais trivial, iterar pelos elementos e como cada bloco tem apenas $\sqrt{n}$ elementos e em uma consulta só pode existir no máximo 2 blocos que não estão inteiramente contidos (Se convença dessa parte), esse caso terá complexidade $O(2 * \sqrt{n})$, gerando uma complexidade total de consulta $O(2 * \sqrt{n} + \sqrt{n}X)$, se **X** não for constante será $O(\sqrt{n}X)$.

Agora precisamos só fazer com que esse **X** seja pequeno, pois se for grande essa solução pode ainda ser pior que a ingênua. Uma ideia seria utilizar um *map*<> do C++, para

contar a frequência dos números, mas o **X** será $O(logN)$,onde o N é o total de elementos, que nesse caso é $\sqrt{n}$, então $O(log(\sqrt{n}))$, o que nos testes que eu fiz ainda não era suficiente, nem com um *unordered_map*<> funcionou. Mas acho que se otimizar bastante o código, *escovando uns bits*, colocando leitura rápida deve passar. Mas isso não é a intenção. Podemos fazer uma *Trie* binária ou na base 10 e guardar todos os números e a complexidade dela é em relação ao tamanho do número. Se utilizarmos a base 2, um número terá no máximo $log_2 N$, onde o **N** é o valor do número e como os números vão até no máximo 10^5, esse valor é aproximadamente 20. Não irei comentar sobre o update, pois como ele é pontual, fica trivial. (Como exercício sugiro tentar o problema com update em range)

A complexidade total é $O(20 * \sqrt{n})$ = $O(logn\sqrt{n})$ por consulta.

Problema C - Coral Nortista
Autor: Cristhian Bonilha
Dificuldade: Médio
Primeiro time a resolver: -
Tema(s): Programação dinâmica

O limite do problema é bem pequeno, então podemos pensar em uma solução bem básica, onde mantemos quais são os 3 cantores que estão no momento e quanto de fôlego cada um ainda tem e a cada momento o cantor de menor fôlego irá sair e entrará um novo e esse novo pode ser qualquer cantor que não esteja já cantando. Selecionamos a combinação que gera o menor tempo.

$F(a,\ b,\ c,\ x,\ y,\ z)$ = O menor tempo para todos os cantores pararem de cantar juntos, sendo *a*, *b* e *c* os índices dos cantores e *x*, *y* e *z* o quanto de fôlego cada um tem, respectivamente. A solução estará na melhor combinação de *a*, *b* e *c*.

É fácil ver que podemos sempre manter ordenado os índices em relação ao fôlego restante sem perder generalidade, assim, o índice *a* sempre será o com menor fôlego.

$$F(a,\ b,\ c,\ x,\ y,\ z) = 0,\ se\ x = y = z$$

$$F(a,\ b,\ c,\ x,\ y,\ z) = min\{min(\{v[i],\ y-x,\ z-x\}) + F(i,\ b,\ c,\ v[i],\ y-x,\ z-x)\}\ \forall i \in \{a,b,c\}$$

Nessa parte a gente pode fazer a reordenação dos índices.

A complexidade fica $O(n^4 10^3)$, parece ser muito ruim mas muitos casos não são explorados e com essa ordenação de índices encontramos ainda mais casos sobreposto, assim é suficiente para passar a questão.

Problema D - Dividindo Círculos
Autor: Francisco Arcos Filho
Dificuldade: Difícil
Primeiro time a resolver: Você é fraco te falta O(1)
Tema(s): Geometria, Combinatória

Este problema podemos resolver através de indução, pensando em quantos novos segmentos serão criados e quantas áreas são divididas ao inserir um novo ponto. Não irei entrar em muitos detalhes aqui pois existe uma demonstração bem completa no Wikipedia.

Esses casos dados no problema eram bem “safados”, levava ao erro de 2^{n-1}, para não tomar um wa sem motivo era só ver que 2^{1000} não cabe em nenhum tipo de dados nativo de C/C++.

Complexidade: $O(1)$

Problema E - Estados do Norte
Autor: Cristhian Bonilha
Dificuldade: Fácil
Primeiro time a resolver: Time ESTACIO 07
Tema(s): Adhoc

Talvez o problema mais fácil da prova, bastava comparar o nome dado com as 7 regiões fornecidas na descrição.

Complexidade: $O(1)$

Problema F - Ferrovias
Autor: Francisco Elio Parente Arcos Filho
Dificuldade: Médio
Primeiro time a resolver: -
Tema(s): Grafos, Floyd-warshall

Em resumo o que o problema nos fornece é a matriz de distâncias após a execução do Floyd-Warshall e temos que informar quantas arestas são necessárias ter no grafo para que ao executar o algoritmo aquela seja a matriz.

O primeiro passo é identificar a matriz inválida, temos que verificar 3 coisas:

- $d[i][j] != d[j][i], \forall i, j$, como as arestas são bidirecionais e se trata de um grafo simples, a matriz tem que ser simétrica;
- $d[i][i] != 0, \forall i$, o custo para sair de um vértice e ir pra ele mesmo tem que ser 0.
- $d[i][k] + d[k][j] < d[i][j] \forall i, j, k \& j != k \& i != k$, como estamos partindo do princípio que foi executado o algoritmo de Floyd-warshall, não pode existir uma forma de minimizar a matriz, ela tem que ser mínima.

Após verificar a matriz inválida basta a gente decidir quantas arestas são necessárias, podemos assumir de início que todas são necessárias, com exceção de *self-loops.* Uma aresta $i-j$ deixa de ser necessária quando existir outras duas arestas $i-k\ e\ k-j$ que a soma delas for igual a de $i-j$. Portanto basta verificar para todo par de vértices se isso acontece. Só tomar cuidado para não imprimir aresta duplicada, pois $i-j = j-i$. No final a resposta é esse total de arestas * K.

Complexidade: $O(n^3)$

Problema G - Binarizando a Matriz
Autor: Gabriel Duarte
Dificuldade: Médio
Primeiro time a resolver: -

Tema(s): Programação dinâmica

A primeira coisa a se perceber nesse problema é que pode ter casos onde a entrada já é inválida, portanto podemos primeiro identificar isso e já imprimir 0.

Agora podemos utilizar algumas observações gulosas bem simples:

- Ao encontrar uma posição (i, j) que tenha o valor 1, só existe uma forma de pintar todas as casas de $(0, 0)$ até ela, que é colocar 1 também. Portanto, fazemos isso.

- Ao encontrar uma casa (i, j) que tenha o valor 0, só existe uma forma de pintar todas as casas $x, y \; onde \; x >= i \; \& \; y >= j$, ou seja, colocar 0. Pois se fosse colocado o 1 iria quebrar a primeira regra do jogo.

A matriz do primeiro exemplo ficaria da seguinte maneira após realizar as mudanças:

1	.
1	.
.	0

Após realizar todo o preenchimento da matriz com os valores triviais, temos que pensar em uma maneira de pintar as células restantes. Para isso modelamos uma programação dinâmica que utiliza os mesmos princípios gulosos citados.

$F(i, j)$ = Quantidade de maneiras de pintar a matriz que começa na linha *i* e as colunas variam de *0* até *j*

O caso base é quando $i = n \; ou \; j < 0$, nesse caso terminamos de pintar e podemos retornar 1. Caso contrário temos 2 opções:

- Assumir que em todas as casas disponíveis para preencher da linha *i* iremos colocar o valor 1, nesse caso podemos diminuir uma linha da matriz sem quebrar nenhuma restrição. Portanto, $F(i + 1, j)$;

- Podemos tentar colocar 0 nas posições vazias daquela linha e ao colocar 0 em uma célula (i, j) nos sobra a possibilidade de pintar a matriz $(i + 1, j - 1)$.

Como queremos o total de possibilidades, basta somar todas essas transições válidas.

Complexidade: $O(n^3)$

Problema H - Ciffra Affine
Autor: Francisco Elio Parente Arcos Filho
Dificuldade: Fácil/Médio
Primeiro time a resolver: Dijkstraidos
Tema(s): Matemática, teoria dos números, inverso multiplicativo

Este problema não é tão difícil de pensar, basta fazer umas operações algébricas. Seja X o valor da letra após a cifragem, sabemos que ela foi obtida através de:

$X = (Y * A + B) \% T$

O que queremos então é o valor de Y, basta a gente separar os termos:

$X = Y * A\%T + B\%T \; -> \; X - B\%T = Y * A\%$, seria então só dividir $X - B\%T$

por A, mas como ele está em módulo, a gente sabe que $(A/B)\%T\ !=\ (A\%T)/(B\%T)$, para realizar essa divisão temos que fazer o inverso multiplicativo, chegando no final em:

$((X - B\%T + T) * inverso(A, T))\%T$

Isso resolve a cifra para cada letra, agora só precisamos saber quando é ambígua, e não é difícil ver que isso acontecerá quando *A* e *T* não forem coprimos.

Complexidade: $O(n)$

Problema I - Ingredientes Típicos
Autor: Cristhian Bonilha
Dificuldade: Fácil/Médio
Primeiro time a resolver: CCL UEA-EST 01
Tema(s): Adhoc, simulação

Este é um problema fácil mas que precisa de atenção. Basta a gente manter uma lista com tudo que é típico e para cada entrada verificamos se a maioria dos ingredientes é típico. Caso a entrada seja típica ela deve entrar na lista de típicos, pois este prato pode ser ingrediente de uma outra entrada. Só tomar cuidado pois um ingrediente pode aparecer em uma entrada e esse ingrediente se "tornar" típico em uma entrada que está mais à frente, isso pode ser visto no segundo caso de teste.

Complexidade $O(50M^2log)$ - *Eu sei, fiz com uma complexidade horrível, mas passou :P*

Problema J - SIBI_Xor
Autor: Francisco Elio Parente Arcos Filho
Dificuldade: Difícil
Primeiro time a resolver: -
Tema(s): Programação dinâmica

Na minha opinião um dos problemas mais difíceis da prova. Para podermos abstrair melhor o problema, vamos separar as operações em três tipos:

1 - Realizar o AND(&) dos elementos;

2 - Realizar o XOR($\oplus$) dos elementos;

3 - Realizar a soma dos conjuntos da resposta para cada problema de tamanho $i\ \forall\ i\ \leq n$

Podemos pensar em problemas desconexos no começo e tentar resolver o problema para um conjunto de tamanho *i* separadamente, pois se tivermos a resposta de todos os tamanhos possíveis, ao final é só realizar a operação do tipo 3 que teremos a resposta do problema.

Vamos recorrer a programação dinâmica, onde:

$F(i, j)$ = Qual a resposta para o problema (Utilizando apenas operações 1 e 2) já tendo passado por *i* itens e com um tamanho *j*. Assim, a resposta para um problema de tamanho *j* estará em $F(n, j)$ e a resposta final do problema é $\sum_{i=1}^{n} F(n, i)$. Podemos fazer dois

casos base, para conjuntos de 0 elementos que é 0 e para conjuntos de 1 elementos que nesse caso não precisamos realizar a operação 1.

$$F(i,\ 0)\ =\ 0$$

$$F(i,\ 1)\ =\ v[i] \oplus F(i-1,\ 1)$$

Agora precisamos resolver para tamanho maiores que 1, o que não é muito difícil, pois quando estamos em um elemento *i* e queremos resolver o problema de tamanho *j* só temos a possibilidade de incluir ele em um conjunto de tamanho *j-1* realizando a operação do tipo 1 ou não incluímos ele, mantendo o conjunto de *i-1* itens com tamanho *j*, agora fazendo a operação 2. O que nos leva a:

$$F(i,\ j)\ =\ (F(i-1,\ j-1)\ \&\ v[i])\ \oplus\ F(i-1,\ j)$$

Isso resolve o problema para todos os tamanhos possíveis, agora basta a gente somar toda a linha *n*. Só tomar cuidado pois o tipo de dados correto para utilizar em C/C++ é o *unsigned long long int* e lembrar de tirar os módulos de $F(n,\ i)$ antes e depois realizar a soma pois é fácil ter *overflow*.

Complexidade: $O(n^2)$

Problema K - K-ésimo Caminho
Autor: Gabriel Duarte
Dificuldade: Médio
Primeiro time a resolver: -
Tema(s): Grafos, Dijkstra

O algoritmo de Dijkstra tradicional "carrega" de informação apenas o vértice que ele está. Porém, para esse problema podemos carregar também quantas arestas que não estão no menor caminho a gente já passou, assim teremos uma matriz de distâncias $dist[i][j]$, que indica qual o menor caminho do vértice de saída até a o vértice *i* utilizando EXATAMENTE *j* arestas diferentes.

Uma coisa a se notar é que cada consulta temos que saber o menor valor para chegar no destino utilizando PELO MENOS **K** arestas, ou seja, podemos usar qualquer valor maior que **K**. Com isso, podemos processar a matriz de distâncias para na posição $(i,\ j)$ ter o valor do menor caminho até *i* utilizando PELO MENOS *j* arestas. Isso pode ser feito da seguinte maneira:

$$dist[i][j]\ =min\ (dist[i][j],\ dist[i][k])\ \forall\ i,\ j\ \&\ k\ >\ j$$

Agora para responder uma consulta basta ver se:

$$abs(dist[destino][0]-dist[destino][k])\ <=\ d$$

Complexidade: $O(nklog)$ - Pode ter um outro fator dependendo de como será feito para verificar se uma aresta está ou não no menor caminho. Eu usei um *set*<> , então tem mais um $log()$ multiplicando ali.

Problema L - Combate à Dengue 2.0
Autor: Gabriel Duarte
Dificuldade: Médio/Difícil
Primeiro time a resolver: -
Tema(s): Programação dinâmica, Bitmask, TSP

Este problema é uma variação do clássico problema do Caixeiro Viajante, irei assumir que o leitor já saiba a versão tradicional.

Em resumo, este problema seria um caixeiro tendo vários sources, e a resposta é o menor caminho para visitar todos os vértices podendo utilizar ou não esses sources.

A solução que eu pensei é uma *dp* que carrega qual vértice estamos, um bitmask que representa quais focos da dengue já foram visitados, quais amigos já foram utilizados e qual o amigo que está fazendo o trajeto atual. O caso base é quando já visitamos todos os pontos, nesse caso retornamos a distância do amigo atual até sua casa (Como o problema deixa claro, todos devem voltar para suas casas). Caso contrário, temos a opção de retornar o amigo atual para sua casa e fazer com que um novo amigo que ainda não participou comece um novo trajeto, ou temos a opção normal do problema do caixeiro viajante, escolher um novo vértice não visitado e ir para ele. Pegamos o menor valor de todas essas opções.

Complexidade: $O((n+m)* 2^{n+m} * n * m * 2^n))$

Problema M - Mais Cavalos
Autor: Francisco Elio Parente Arcos Filho
Dificuldade: Fácil
Primeiro time a resolver: CCL UEA-EST 01
Tema(s): Adhoc

Este problema é bem fácil, basta a gente ver que qualquer movimento do cavalo ou se mexe em duas unidades da vertical e uma na horizontal ou uma na vertical e duas na horizontal. Se as posições forem strings basta fazer:

Se $abs(a[0] - b[0]) == 1$ && $abs(a[1] - b[1]) == 2$
    print VALIDO
Senão Se $abs(a[0] - b[0]) == 2$ && $abs(a[1] - b[1]) == 1$
    print VALIDO
else
    print INVALIDO

Complexidade: $O(1)$